

From optical sorting complexity to hybrid photonic mappings

Master's thesis: research-phase documentation

Bharadwaj Srikumar | Revision 6 | 2 October 2026

$C = (T, S, H, D)$

Source-grounded architecture comparison

Explicit 4-bit and 8-bit numerical evaluation

This document preserves the established historical-model comparison and the four-parameter derivations for five optical/hybrid sorting architectures. It updates the modern bitonic/rank mapping, validation, numerical assumptions and results using the protocol agreed on 2 October 2026.

Current milestone

216 configurations; 216,000 architecture executions; 24,000 generated input arrays. Every ideal case sorts correctly and stably. Finite signed-output resolution causes false ties, while inconsistent pair decisions can invalidate ranks.

The study supports thesis registration through a reproducible comparison and explicit hardware questions. Q.ANT device execution, calibrated optical cascades and a complete energy-efficiency verdict remain future work.

This revision supersedes the earlier precision sweep. Ranges are now 0..15 and 0..255, input errors are rejected, and the two quantizer widths are separate configuration fields.

Contents

Chapter	Page
1. Executive research position	3
2. Concepts needed to read the study	4
3. The four parameters: T, S, H and D	6
4. Historical models and their relationship to C	8
5. Five sorters compared with $C=(T,S,H,D)$	10
6. Why bitonic versus rank is the focused thesis comparison	17
7. Q.ANT: what its capabilities mean for sorting	18
8. One difference operation, two sorting mappings	21
9. What was validated against the papers	29
10. The agreed experimental protocol	31
11. Results: what the new run establishes	33
12. Next experiments and hardware questions	40
13. Thesis registration: a concrete research goal	41
14. Reproduce and inspect the evaluation	42
15. Glossary and evidence boundary	44

Charts and exact counts are rebuilt from the saved 2 October evaluation. Historical foundation and hardware evidence retain their dated source boundaries.

1. Executive research position

The thesis asks when a photonic/hybrid sorting mapping makes sense under explicit correctness, resource and execution requirements. It does not assume an advantage over digital hardware.

Question	Current answer
What has been validated?	Selected source equations and examples; ideal functional behavior; quantizer controls; a complete new 4/8-bit sweep.
What are the modern mappings?	Pair differences plus electronic compare-exchange for bitonic; pair differences plus electronic ranks and placement for rank.
Where do they fail?	Adjacent keys may round to the same difference code; noise changes decisions; rank can develop collisions.
What remains a hardware question?	Matrix dimensions, sign accuracy, transfers, nonlinear functions, optical chaining and regeneration.

Beyette and Desmulliez are retained as separate source architectures. Under the shared numerical model they execute the same fixed-shuffle schedule and receive identical comparison-error streams. Their equal precision results therefore do not distinguish physical devices.

The numerical evidence compares two operating scenarios: 4-bit keys with 4-bit input/output conversion, and 8-bit keys with 8-bit input/output conversion. Domain size changes with precision. A future controlled sweep should hold keys fixed while varying converter settings independently.

The preceding theoretical chapters are preserved. Subsequent numerical chapters and results are replaced, rather than mixing old 6..9-bit results with the new protocol.

2. Concepts needed to read the study

Sorting, records and correctness

A key is the value used to order data. A record contains that key and associated information, such as a payload or original position. Full sorting returns every record exactly once in order. Stable sorting additionally retains the original order of equal keys. Ranking assigns positions; a full sorter must also realize the output placement. Top-k retrieval returns only a selected subset and is a different task.

For example, records (8,A), (2,B), (8,C) must become (2,B), (8,A), (8,C) under stable ascending sorting. Preserving values alone does not establish that record identities or duplicate order were preserved.

Digital, analog, optical and hybrid

Digital computation represents discrete states, often bits. Analog computation represents numerical quantities through physical signals. Optical describes the physical carrier: an optical system can implement digital Boolean logic or analog arithmetic. Consequently, “optical sorting” does not imply that its comparisons use analog subtraction. The historical bitonic designs and the proposed noisy hybrid adaptation must be distinguished. [4-7]

A hybrid photonic processor divides work between optics and electronics. A typical arithmetic path encodes inputs into light, performs a physical operation, detects the result and performs digital control or further processing. Optical communication alone does not make the arithmetic photonic; the FPGA radix sorter is an important example. [8]

Parallelism, work and dependency

Total work counts all primitive operations. Parallel time counts sequential rounds once dependencies and available resources are fixed. A round can contain many independent operations. Hardware reuse saves simultaneous processing units but may increase rounds. A pipeline overlaps different datasets; a single dataset still traverses its required dependencies.

Fan-in is the number of inputs combined by an operation. Fan-out is the number of destinations receiving an input. Broadcasting to N destinations and summing N inputs are substantial physical requirements even when an abstract model counts each as one stage. The rank sorter makes this distinction particularly visible. [2,7]

Asymptotic notation and physical units

$O(f(N))$ states an asymptotic upper bound; $\Omega(f(N))$ states a lower bound; $\Theta(f(N))$ means matching upper and lower order. Constants and implementation details are suppressed. $\Theta(\log^2 N)$ does not specify nanoseconds, and $\Theta(N^2)$ hardware does not specify square millimetres. All logarithms in the sorting formulas below are base two.

Latency is time until one dataset finishes. Initiation interval or output period is the time between successive datasets in steady operation. Throughput is completed keys or bytes

per second. Power is energy per unit time; energy per sort requires an agreed system boundary and workload duration. These quantities cannot be replaced by one another.

Physical limits that motivate the numerical study

A modulator encodes an electrical input onto an optical signal. A photodetector converts arriving light into an electrical response. A digital-to-analog converter (DAC) produces an analog drive from digital data; an analog-to-digital converter (ADC) samples and quantizes an electrical result. The converter, optical path and electronics can each limit precision. An API receiving floating-point arrays does not reveal the resolution of each internal component. [13-15]

Optical loss reduces useful signal power during propagation, splitting, coupling and component traversal. A fan-out network divides available optical power among destinations unless additional power or amplification is supplied. Longer unregenerated paths can therefore face both weaker signals and accumulated disturbances. Loss is often reported in decibels: a power loss of L dB gives an output/input power ratio of $10^{-(L/10)}$. This relation alone does not specify the resulting numerical error.

Shot noise is associated with discrete photodetection events. Thermal and electronic noise arise in receivers and related circuitry. Laser relative intensity noise describes fluctuations in source intensity. Crosstalk is unwanted coupling between channels. These effects may depend on signal level, bandwidth, shared components and operating conditions; they need not be independent or identically distributed. The current independent Gaussian comparison-error model does not distinguish or calibrate these mechanisms.

Signal-to-noise ratio (SNR) compares useful signal power with noise power under a stated bandwidth and measurement convention. Effective number of bits (ENOB) expresses achieved signal or numerical quality in bit-like terms under a specified test. Nominal converter resolution merely counts representable codes. No SNR-to-ENOB calibration or physical noise budget has been performed for these sorting mappings.

Quantization is a deterministic discretization, while a random-noise model adds statistical perturbations. Either can make nearby keys indistinguishable. Sorting is especially sensitive near equality because the sign of a small difference decides the order. High average arithmetic accuracy does not guarantee a fully correct ordering of many closely spaced keys.

Techniques such as bit slicing or block floating-point representations can alter precision and conversion workload. Bit slicing decomposes a value into smaller bit groups processed separately; block floating point shares a scale or exponent across a group of values. Neither technique is implemented in the current evaluation. Adding either would require a new operation count, representation rule and error analysis rather than treating it as a free precision improvement.

These physical concepts motivate the later calibration and regeneration questions. They are not extra fitted parameters in the present pilot: the executed model contains only the stated quantizers and independent comparison noise. The distinction prevents an illustrative numerical experiment from being presented as a complete device model.

3. The four parameters: T, S, H and D

The comparison uses the five researched sorting structures and counts how their required steps and resources grow with N . It retains all four parameters of $C=(T,S,H,D)$. Device dimensions, clock rates, bit-serial delays, converter costs, optical losses and measured power are excluded from these complexity expressions.

Parameter	What is counted	Simple question
T: time	Dependent processing rounds needed to finish one sort, with the parallel operations provided by the stated structure.	How many rounds must follow one another?
S: space	Peak live logical storage for one input batch, including input, output and necessary intermediate data.	How many values or logical cells must coexist?
H: hardware	Processing-unit positions present at the same time in the stated structure; reused units are counted once.	How many processing positions are provided?
D: optical depth	Longest sequence of optical stages traversed before signal restoration. Electronic decision/storage followed by a fresh optical output ends a segment.	How many optical stages can one signal traverse without restoration?

The counting units remain simple

N is the number of records. S is reported in logical words/cells so that a record, index or rank needs $O(1)$ words. This is the already adopted N -only normalization of the original bit-based S ; it does not change S into area. H counts word-processing positions, not transistors or square millimetres. D counts optical blocks or links, not nanoseconds, metres or individual lenses.

Retain the sorting structure; omit device-specific costs

The distinction between a spatial pipeline and reused processing arrays is part of the researched structure. Keeping that distinction lets H remain useful. Similarly, D uses the optical/electronic boundaries shown in the source architecture. Removing those structural facts as well would make H and D undefined; that is not the purpose of this comparison.

Thus the table is an algorithmic comparison of the five published structures. It is not a claim that every possible implementation of the same sorting algorithm has identical H or D . No additional binary reduction circuit or undocumented photonic cascade is introduced.

3.1 Rules used consistently in the table

Rule	How it is applied
One whole-key operation	A compare-exchange, arithmetic operation or record move has unit cost. Physical key serialization and converter precision are evaluated later.
Parallel operations	Independent operations in a source stage happen concurrently. Louri's original broadcast, wide column sum and parallel output placement remain source-level primitives.
One input batch	S counts live data for one sort. Installed registers for many overlapping pipeline batches and stored copies of the entire schedule are not added to this logical-space count.
Processing units versus memory	H counts processing positions. Memory capacity is recorded by S, so fixed processing resources can coexist with growing S. A whole $N \times N$ array is not counted as one scalar processing unit.
Optical path boundaries	D restarts at data restoration, including digital data decisions followed by fresh optical encoding. Restoring only the comparison control does not restore a passively routed payload.
N-only radix row	Keep key format, digit range and the number of processing streams fixed while increasing N. This is stated beside the row; growing the cluster is a separate scaling question.

Only two source qualifications need special attention

Pipelined optical C&E: the paper explicitly gives passive and restoring exchange variants. T, S and H have the same orders; D differs. Both D values appear in the same table cell rather than silently choosing one. [4, p.1725, Figs.5-6]

Radix: a single-variable result requires a declared resource scaling rule. The row uses a fixed number of processing streams and the paper's fixed key format. It does not choose a particular FPGA clock, board model or optical bandwidth. Without holding the processing resources fixed, T and H would require another variable. [8]

Notation

For bitonic sorting, $m = \log_2 N$, $K = m(m+1)/2$ is the number of active compare-exchange layers, and $P = m^2 - m + 1$ is the researched fixed-shuffle schedule length, including bypass steps. K and P are derived from N. For radix, q is the number of digit passes and R is the digit range; both are constants in its table row.

$O(f(N))$ describes an asymptotic upper bound. The derivations below establish the stated growth orders under these counting rules. These are structural complexity results, not Q.ANT timing or energy predictions. T counts rounds throughout the main table; there is no separate sequential-time column.

4. Historical models and their relationship to C

Thompson: area-time complexity for VLSI (1979)

Thompson studies physical chip area and computation time in a VLSI model, using the discrete Fourier transform as the central problem. The first page of the supplied paper states an area-time-squared lower bound, $AT^2 \geq N^2/16$, within its model. Wire layout and communication matter; the number of active arithmetic elements alone does not determine physical size. [1, p.81]

The connection to this thesis is the trade-off between spatial hardware and execution time. However, Thompson's physical area A is neither logical storage S nor a raw component count H . The DFT bound is not imported as a sorting lower bound. To turn H into area, a separate model for cell dimensions, wires, optical routing, memory and peripheral circuits would be required.

Caulfield: space-time complexity in optical computing (1991)

Caulfield describes normalized spatial, temporal and fan-in resources. Under the paper's operation definitions, the relation $X_A X_T X_F \geq X_P$ expresses that area, time and fan-in together must support the required computational work. X_P denotes the problem-operation measure, not this document's bitonic processing-step symbol P . [2, pp.373-378]

This is directly relevant to rank sorting: many pair comparisons and wide sums can be packed into a small number of abstract stages by allocating many spatial channels and large fan-in. A constant stage count is therefore not free computation. C retains the time/hardware distinction, while fan-in and fan-out remain explicit metadata. No claim is made that the four coordinates alone replace all of Caulfield's assumptions.

Li and Monticone: spatial complexity of optical computing (2025)

This newer work connects physical wave-device dimensions to the structure of mathematical operators, including communication across cuts and overlapping nonlocality. In plain terms, information that must cross a spatial division requires sufficient independent physical channels. Structural sparsity can alter the required device geometry. It is a modern complement to the historical models, not itself an older historical paper. [3]

Its contribution to the present analysis is a warning against equating sparse arithmetic with small total hardware. Bitonic has sparse pairwise operations but must route values between pairings. The whole sorter is nonlinear, so a result for a linear operator cannot be applied unchanged to the complete sorting task. Physical thickness or cross-section is not the same thing as D , the number of unregenerated stages.

Source	Shared insight	Not interchangeable with C
Thompson [1]	Execution time depends on spatial resources and communication.	Physical A $\neq$ logical S; gate count alone does not determine layout area.
Caulfield [2]	Wide parallelism and fan-in can reduce abstract step count.	N-input summation needs its own primitive and precision assumptions.
Li & Monticone [3]	Communication structure constrains physical size.	Device dimensions are not unregenerated optical depth; linear bounds are not full-sort bounds.
Current framework [F1]	Compare T, S, H and D under explicit execution assumptions.	An operational synthesis, not a new universal complexity theorem.

The thesis contribution is the disciplined application of these resource ideas to concrete sorter mappings and reproducible experiments. It does not require claiming the first hardware-aware complexity model or proving a new physical lower bound.

5. Five sorters compared with $C=(T,S,H,D)$

All values use N as the growth variable. T counts dependent rounds, S counts live logical words/cells, H counts processing positions and D counts unregenerated optical stages. Each row is derived separately on the following pages.

Sorter	T	S	H	D
Pipelined bitonic with optical C&E [4]	$O(\log^2 N)$	$O(N)$	$O(N \log^2 N)$	$O(\log^2 N)$ passive; $O(1)$ restoring
Beyette: recirculating bitonic [5]	$O(\log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Desmulliez: perfect-shuffle bitonic [6]	$O(\log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Louri: rank-based broadcast-and-compare [7]	$O(1)^*$	$O(N^2)$	$O(N^2)$	$O(1)$
In-network stable radix [8]	$O(N)^\dagger$	$O(N)$	$O(1)^\dagger$	$O(1)$

* **Rank:** the paper's simultaneous broadcast, comparisons, column sums and output placement are retained. Constant time means a constant number of these parallel stages, with quadratic resources; it does not mean constant total work.

† **Radix:** fixed key format and fixed number of processing streams. The paper uses eight 4-bit passes for 32-bit keys. H counts processing resources, while the memory holding N records is counted separately under S . If the cluster grows with N , this row must be recalculated.

Pipeline D: passive exchange retains the optical data signal across the network; restoring exchange renews it at each stage. The paper describes both. Regeneration of the comparison control alone does not justify $D=O(1)$ for a passive data path.

Communication is included in D: an optical transmission between electronic endpoints is an optical stage. Therefore the FPGA radix row has $D=O(1)$, even though its sorting arithmetic is electronic. If D counted only optical arithmetic, that particular quantity would instead be zero; this table consistently includes optical data transfers.

How to read the comparison

The spatial bitonic pipeline provides all its layers as hardware. Beyette and Desmulliez reuse a linear number of positions. Rank sorting provides a quadratic comparison plane to obtain a constant stage count. The radix row streams the records through a fixed number of processing positions. These are different time/resource trade-offs, not equal-hardware speed measurements.

The next five pages each use the same order: derive T , then S , then H , then D . There is no change to the definitions between sorters.

5.1 Pipelined bitonic sorter: four derivations

Source: Stirik and Athale, Sorting with optical compare-and-exchange modules. The bitonic network is spatially laid out; the paper distinguishes restoring and passive exchange units. [4, pp.1721-1722 and p.1725]

T = O(log²N): count the dependent layers

For $N=2^m$, the merge levels have lengths 2, 4, ..., N. Their respective merge depths are 1, 2, ..., m. All $N/2$ independent compare-exchanges within a layer happen together, but the next layer must receive the previous layer's results.

$$K = 1+2+\dots+m = m(m+1)/2, \text{ with } m=\log_2 N.$$

$$T = O(K) = O(\log^2 N).$$

S = O(N): count the live data for one batch

The current layer contains N records; the next layer replaces them with the same records in a different order. Input/output and temporary values require only a constant number of N-sized collections. A whole-key compare-exchange has constant temporary state. Hence $S=O(N)$ logical words for one batch.

A fully occupied physical pipeline can hold many batches and can have state in every stage. That is installed pipeline storage, not the single-batch logical S used here. Likewise, the paper's spatial module count belongs under H, not S.

H = O(N log²N): count the provided C&E positions

There are $N/2$ word-level C&E positions in each of K spatially provided layers. They remain present even when only one layer is processing a particular batch.

$$H = (N/2)K = N m(m+1)/4 = O(N \log^2 N).$$

D: follow the data through the exchange units

Passive exchange: the payload is routed through K successive exchange layers without data regeneration. Counting a constant number of optical blocks per layer gives $D=O(K)=O(\log^2 N)$. This is the required structural depth, not proof that arbitrarily deep passive cascades remain accurate.

Restoring exchange: the source's regenerative exchange circuit renews the data signal at each layer. The longest segment between renewals contains only a fixed-size stage, so $D=O(1)$. Both cases follow directly from the source's exchange alternatives. [4, Fig.5 versus Fig.6, p.1725]

$$C_{\text{pipeline}} = (O(\log^2 N), O(N), O(N \log^2 N), D),$$

where $D=O(\log^2 N)$ for passive exchange or $O(1)$ for restoring exchange.

5.2 Beyette recirculating sorter: four derivations

Source: Beyette et al., Bitonic sorting using an optoelectronic recirculating architecture. Two optoelectronic processing arrays are reused; digital data are latched and emitted by VCSELs between arrays. [5, Sections 2.A-C]

T = O(log²N): count the repeated processing steps

The fixed-shuffle schedule requires $P=m^2-m+1$ processing steps for $N=2^m$ records. Each step applies a word-level operation to the array in parallel. Treating each such operation as unit cost gives:

$$T = O(P) = O(m^2-m+1) = O(\log^2 N).$$

The original bit-clock expression includes word length and overlap between bit operations. Those details are retained in the historical timing validation. They are not additional variables in this word-level complexity table. [5, Eq.(1) and Section 3.C]

S = O(N): count the records and current state

Each of the two reused arrays holds a batch of N word positions. Their data, current comparison decisions and control state require a constant number of linear collections at word granularity. Thus $S=O(2N+N)=O(N)$. The schedule can be generated as execution proceeds; all P copies of it need not be stored.

H = O(N): count physical positions only once

Each array contains $N/2$ pair-processing positions at word granularity. A fixed number of such arrays provides a total proportional to N . Reusing a position in P rounds does not create P new hardware positions.

$$H = O(2 \times N/2) = O(N).$$

The source constructs these positions from bitwise modules. Counting their bit-level components would reintroduce word length. This table consistently uses word-processing positions for H .

D = O(1): each digital transfer ends the optical segment

The source detects the incoming optical data, processes/latches digital signals and drives new VCSEL output. Its discussion also states that the output can restore losses in the optical path. Thus a signal segment traverses the fixed inter-array optical connection before digital restoration and fresh emission. [5, Section 2.B, p.8167; Section 2.C, p.8169]

Increasing N increases P , the number of times this structure is used. It does not concatenate those separately restored transfers into one unregenerated path. Hence $D=O(1)$, even though $T=O(\log^2 N)$.

$$C_{\text{Beyette}} = (O(\log^2 N), O(N), O(N), O(1)).$$

5.3 Desmulliez perfect-shuffle sorter: four derivations

Source: Desmulliez et al., Perfect-shuffle interconnected bitonic sorter: optoelectronic design. The sorting-node array and memory are reused; data pass through electronic storage and nodes driving fresh optical read beams. [6, Sections 2, 3.C and 4.A-B]

T = O(log²N): count the fixed-shuffle schedule

The first merge uses one processing step. Each of the remaining m-1 merges uses m steps, including bypass steps needed for the fixed interconnection. Therefore:

$$P = 1 + m(m-1) = m^2 - m + 1.$$

$$T = O(P) = O(\log^2 N), \text{ because } m = \log_2 N.$$

The compact network's $K = m(m+1)/2$ active compare-exchange layers and the source's P processing steps are different exact counts, but both have order $\log^2 N$. The source's frequency and bit-serial timing formula are not included here. [6, Section 3, pp.5079-5081]

S = O(N): count data and current masks

The memory array holds N words. Sorting-node state and the current control masks occupy a linear number of logical positions. The source generates successive masks through its regular shuffle procedure, rather than storing a separate complete mask for every step. A constant number of N-sized collections gives $S = O(N)$. [6, Section 3.B-C]

H = O(N): count the reused sorting nodes

The paper explicitly gives $N/2$ sorting nodes for an N-input arrangement. Current control and memory-interface processing add only a constant amount per word position. The node array is reused through all P steps.

$$H = O(N/2 + N) = O(N).$$

D = O(1): use the data-path restoration boundary

The source's shuffle instruction reads a bit plane from memory, sends it through the optical shuffle and sorting nodes, and stores the node output back in memory. Its nodes receive light at detectors, process/store the data electronically and drive output modulators on further read beams. [6, Section 3.C and 4.A-B, pp.5081-5082]

Thus successive loop traversals are separated by digital data storage and renewed optical output. Only a fixed number of optical blocks belongs to one unregenerated segment, giving $D = O(1)$. This conclusion uses the source's data path; the mere existence of an electronic control mask would not by itself establish data regeneration.

$$C_{\text{Desmulliez}} = (O(\log^2 N), O(N), O(N), O(1)).$$

5.4 Rank-based broadcast-and-compare: four derivations

Source: Louri et al., Constant-time parallel sorting algorithm and its optical implementation using smart pixels. Keep the original matrix broadcast, wide column sums and parallel output-selection operations. [7, Section 2 and Section 3]

T = O(1): count the source's fixed sequence of stages

The algorithm broadcasts the inputs, forms differences, resolves ties, thresholds comparisons, sums columns into ranks and places records at their output positions. All $N \times N$ entries in a matrix stage are processed simultaneously. The paper lists seven main steps, including a separate bias-generation step; their number does not grow with N . [7, pp.3088-3090]

$T = O(1) + O(1) + \dots + O(1)$, for a fixed number of stages,
so $T = O(1)$.

Column summation is retained as the paper's wide parallel primitive. Replacing it with a binary tree would change the counting model and produce a different bound. Final output placement is included: this is full sorting, not only rank calculation.

S = O(N²): count the matrix data

The described procedure represents a constant number of $N \times N$ difference, comparison and output-selection matrices. The N inputs, N ranks and N outputs are smaller additional collections. Therefore:

$S = O(N^2 + N) = O(N^2)$.

This is the source matrix procedure's logical space. A sequential program that immediately adds each comparison into a rank counter can use less storage, but it no longer represents all source comparisons simultaneously.

H = O(N²): count the spatial processing positions

The $N \times N$ smart-pixel/comparison plane provides a quadratic number of parallel positions. Output selection requires at most another quadratic plane; rank channels add only $O(N)$. A constant number of planes therefore gives $H = O(N^2)$. The whole plane is not counted as one scalar processor.

D = O(1): the optical sequence does not grow deeper

Increasing N expands the planes and the width of the column sums. It does not add sequential optical stages to the source's fixed broadcast/threshold/rank/output sequence. The maximum optical segment is therefore bounded by a constant, giving $D = O(1)$. Smart-pixel detection and re-emission can divide that fixed sequence further, without changing the asymptotic order. [7, Figs.1-5]

This is a structural depth bound. It does not assert that fan-out power, wide-sum precision or optical dimensions remain independent of N . Those are questions for the physical evaluation.

$C_{\text{rank}} = (O(1), O(N^2), O(N^2), O(1))$.

5.5 In-network stable radix sorter: four derivations

Source and scaling rule: Mizutani et al., In-network stable radix sorter using many FPGAs with high-bandwidth photonics. Increase the record count N for a fixed key format and fixed number of processing streams. The particular board count, frequency and link bandwidth are not needed for the resulting N -only orders. [8]

$T = O(N)$: count the work in the digit passes

A stable counting pass scans N records, forms a histogram over R digit values, computes bucket positions and places the records. Its operation count is $O(N+R)$. There are q digit passes. With a fixed number of processing streams, parallel processing changes constant factors rather than the order in N .

$$T = O(q(N+R)).$$

The source task has $q=8$ passes and $R=16$ digit values:

$$T = O(8(N+16)) = O(N).$$

This abstracts each record-processing/transfer step as unit cost on the fixed structure. The paper's eight passes are not eight scalar time steps: each pass must process the records. [8, Section 2.C-D, Table 1 and Section 3]

$S = O(N)$: count record buffers and counters

Input/output buffers hold $O(N)$ records. The fixed digit range and fixed processing structure require a constant number of histogram/bucket counters and current control values. Hence $S=O(N+R)=O(N)$. Counters up to N occupy one logical word under the stated word convention.

$H = O(1)$: count the processing positions being reused

The number of streams and their counting/routing units is held constant as N grows. More records reuse those positions for longer; they do not require one new processing unit per record. Thus $H=O(1)$. Growing memory capacity is represented by $S=O(N)$, not hidden in a claim of constant total device area.

$D = O(1)$: count one optical link segment

Sorting arithmetic and in-network processing are electronic. Optical transceivers connect electronic FPGA endpoints through the source's communication network. Receiving and electronically forwarding data separates successive optical transmissions. Each optical segment contains a fixed number of link/routing blocks in the stated fixed network, so $D=O(1)$. [8, Section 2.B and Figs.2-3]

This D includes optical communication. It is not a claim of optical radix arithmetic. Increasing the number of transmitted records or digit passes increases time/traffic, not the longest segment between electronic endpoints.

$$C_{\text{radix}} = (O(N), O(N), O(1), O(1))$$

for fixed key format and fixed processing resources.

5.6 Reading the four parameters together

A small bitonic example: $N=16$

Quantity	Value	Parameter it explains
Merge levels	$m=\log_2 16=4$	Used to derive the processing depth.
Active compare-exchange layers	$K=4 \times 5/2=10$	T for the compact bitonic network.
C&E positions in a spatial network	$(16/2) \times 10=80$	H counts all provided layer positions.
C&E positions in one reused layer	$16/2=8$	H stays linear when a fixed number of arrays is reused.
Source fixed-shuffle schedule	$P=4^2-4+1=13$	T uses P for those schedules; P differs from K.
Live records for one batch	16 plus linear temporary state	S is linear, independently of spatial replication.

Why T and D are not the same

Ten dependent layers require ten rounds for that batch. If the data signal is restored after every layer, however, it never traverses ten layers without restoration. T can therefore grow as $\log^2 N$ while D stays constant. A passive payload path instead retains the signal across the layers and has growing D. This distinction is the reason for retaining D in the framework.

Why S and H are not the same

The spatial bitonic network provides all its comparison positions, but a single batch still consists of only N live records plus linear temporary state. Rank sorting explicitly represents quadratic matrix data as well as quadratic processing positions, so both S and H are quadratic there. Memory and processing need separate counts.

What this comparison establishes

The table compares source structures through one set of definitions. It identifies depth, storage and processing-resource trade-offs. It does not identify a universal speed or energy winner: the rows provide different amounts and types of parallel processing. On a fixed modern photonic core, the mappings must account for how those source operations are scheduled onto the available resources.

For context only, sequential execution has $O(N \log^2 N)$ work for bitonic and $O(N^2)$ for pairwise rank sorting. These work counts do not replace T in the main table. The paper's wide rank primitive also does not promise a constant-time operation on an ordinary sequential RAM.

6. Why bitonic versus rank is the focused thesis comparison

The five-architecture review is the breadth of the literature foundation. The experimental core is narrower: compare two algorithmic organizations on a common candidate arithmetic interface. Bitonic creates many dependent rounds but fewer pair evaluations. Rank exposes more independent pair work, followed by rank accumulation and placement. Neither is assumed preferable before resource and correctness constraints are specified.

Bitonic work: $NK/2 = N m(m+1)/4$ pair evaluations.

Pairwise rank work: $N(N-1)/2$ pair evaluations.

For $N=32$, bitonic has 15 active rounds and 240 pair evaluations; rank has 496 independent pair evaluations before summation and placement. Rank's pair phase is wider, but finite device capacity can force several batches. One API call does not establish constant physical time.

Compare-exchange is nonlinear because the destination depends on the ordering of the inputs. A linear MVM can calculate $a-b$, but a decision and routing are still needed. The mathematical identity $r=\max(a-b,0)$, $\min(a,b)=a-r$ and $\max(a,b)=b+r$ motivates a possible nonlinear formulation. It does not establish that a public photonic API implements optical ReLU or arbitrary stage cascading.

The baseline keeps original records electronically and uses candidate accelerator arithmetic for differences. This is deliberately modest and inspectable. The next major scientific extension is native optical nonlinearity and regeneration, if a defensible function and signal path can be modeled. The extension cannot be presented as a Q.ANT capability unless documentation or measurements support it.

PRISM is included as related work because it uses photonic similarity scoring in a selection workflow. Its electronic top-k block selection for a retrieval task is not a full sorting network, and its task-specific efficiency cannot be transferred to returning every record in sorted order. [12]

7. Q.ANT: what its capabilities mean for sorting

The task is to sort records, but the candidate accelerator accepts numerical operations. The mapping must therefore identify which part of sorting can use those operations, which parts remain electronic, and what the public evidence actually establishes. Product facts were rechecked on 1 October 2026; software statements use the pinned source snapshot in [10].

Evidence	Meaning for this study	What still needs checking
LNol/TFLN photonic core [9]	Electrically controlled optical circuits can evaluate arithmetic functions.	The exact internal circuit used for the proposed difference operation.
Linear API: $Y=XW^T$ [10]	A pair difference can be expressed with weights +1 and -1.	Device support and efficient execution for two inputs and one output; accepted ranges, padding and tiling.
Periodic nonlinear API [10]	A separate driver-backed function computes $\text{tcos}(u) \cdot v$.	A calibrated comparator or min/max mapping; direct optical chaining.
Python/C interfaces; PCIe Gen4 x8 [9]	Digital arrays can be exchanged with a host application.	Dispatch, transfers, retained weights and device-resident buffers.
8 GOPS nonlinear throughput; 150 W NPU listing [9]	Vendor operating specifications to record with their workload and boundary.	Pair differences per second and measured power for sorting. These figures do not establish either.

Why a subtraction is the first candidate

- Sorting asks whether a is greater than b . The sign of $d=a-b$ answers that question; the difference is a linear operation.
- The decision “is d positive?” is nonlinear. Sending a difference through an MVM does not also perform that decision or exchange the records.
- The pilot therefore separates arithmetic from control: model the difference, then compare its sign and move the original records digitally. This makes the first precision experiment interpretable.

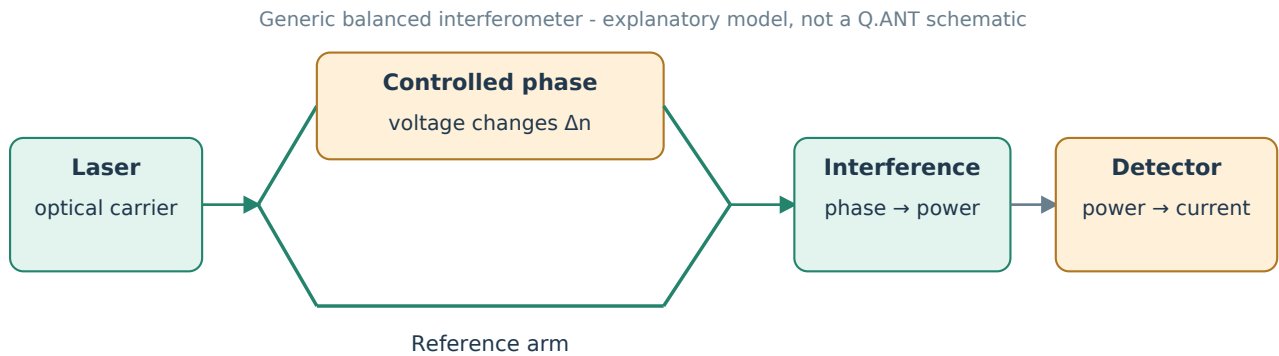
Three kinds of evidence

Documented: public interfaces and vendor specifications. **Calculated:** the sorting equations and logical matrix shapes derived here. **Measured in software:** outcomes of the NumPy pilot. None of these alone is a measurement of the proposed mapping on a Q.ANT device.

Sources: [9] official product page, rechecked 1 October 2026; [10] official SDK at commit 72a2d99f10240b6df3c6d0f636dfa0e2b5d38902. The CPU SDK backend and the NPU have not been run in this evaluation.

7.1 The physics: numbers, light and readout

A digital array does not become optical simply because Python calls a function. Electronic values must control an optical signal; optical transformations then act on that encoding; detection returns an electrical result. The following is a generic explanation of the mechanism, not a reconstruction of Q.ANT's unpublished internal layout.



1. Voltage changes phase

Lithium niobate has an electro-optic response: an applied electric field changes its refractive index. In a waveguide of length L , a change Δn produces the phase change below. λ is the optical wavelength. Over the small-signal Pockels regime, Δn is proportional to the applied electric field; electrode geometry relates that field to voltage.

$$\Delta\phi = (2\pi/\lambda) L \Delta n$$

2. Interference turns phase into an observable signal

Two coherent optical paths recombine. For an ideal balanced, lossless interferometer, one output has $P_{\text{out}} = (P_{\text{in}}/2)[1 + \cos(\Delta\phi)]$. P denotes optical power; $\Delta\phi$ is the relative phase. This periodic response explains how an optical circuit can supply a nonlinear input-output function. It is not automatically a threshold or ReLU, and tcos in the SDK must not be silently replaced by ideal $\cos$.

3. A detector measures power, not a signed real number

In its linear operating regime, a photodiode gives photocurrent $I = \mathcal{R}P$, where $\mathcal{R}$ is responsivity in amperes per watt. Electronics amplify/read this current; digitization produces an array value. Optical power cannot be negative. Signed arithmetic therefore needs an encoding/readout scheme, such as separate positive and negative channels or phase-sensitive detection.

For example, two detected channels can give $I_{\text{diff}} = \mathcal{R}(P_a - P_b)$. This explains physically how a signed difference can be obtained from nonnegative powers. Louri's source explicitly uses two optical channels and electronic subtraction in smart pixels. Which signed-weight mechanism Q.ANT uses for the proposed kernel remains to be confirmed.

Physical background: supplied The Physics of Optical Computing [13]; Louri [7], Section 3.A; and Integrated lithium niobate photonic computing circuit based on electro-optic conversion, Nature Communications 16, 8178 (2025), Fig.1: doi:10.1038/s41467-025-62635-8. The latter is a separate TFLN research circuit, not Q.ANT hardware.

7.2 Precision and nonlinearity: updated evidence

The public Q.ANT evidence in the preceding pages is retained as a dated snapshot checked on 1 October 2026. This revision reruns the numerical experiment on 2 October 2026. It does not establish a new device specification or execute the Q.ANT SDK.

Quantity	Meaning in this study
key_bits = 4 or 8	Declared integer domain: 0..15 or 0..255. It determines input values and the maximum tested N.
input_bits = 4 or 8	Number of bits in the hypothetical unsigned input quantizer. Stored independently of key_bits.
output_bits = 4 or 8	Number of bits in the hypothetical signed difference-output quantizer. Stored independently of input_bits.
BF16	Software storage format at the inspected SDK boundary. It is not emulated by these fixed-point quantizers.
ENOB	Effective precision from a specified measurement. Neither 4 nor 8 is asserted to be Q.ANT ENOB.
Gaussian sigma = 0.25 output steps	An agreed illustrative sensitivity setting; not a measured or paper-derived Q.ANT noise amplitude.

What the software evidence actually supports

At pinned SDK commit 72a2d99f10240b6df3c6d0f636dfa0e2b5d38902, linear_fprop defines $Y=XW^T$ and calls a batched MVM driver. The nonlinear interface describes $\text{tcos}(u)*v$. In the inspected implementation, relu_fprop applies a software maximum rather than demonstrating native optical ReLU. [10]

Supported physical matrix dimensions, small-difference accuracy, data-transfer costs and unregenerated cascade depth remain open questions. Eight channels do not establish an 8 by 8 matrix. Vendor GOPS do not establish sorts per second. The experiment therefore evaluates a proposed mathematical mapping with explicit numerical assumptions.

8. One difference operation, two sorting mappings

Both mappings need to decide which of two records comes first. Their common candidate arithmetic is a signed subtraction. For B independent pairs, gather a B by 2 input matrix and use one output weight row $[1, -1]$.

$$X = [[a_1, b_1], \dots, [a_B, b_B]], W = [1, -1], Y = X W^T.$$

The multiplication produces differences, not sorted records. A sign decision, tie rule and record movement still follow. The code computes this subtraction directly in NumPy; it does not pretend that a Python matrix expression invokes an optical device.

1. Electronic

Gather current pairs

2. Candidate photonic path

Encode / difference / read

3. Electronic

Sign and tie rule

4. Electronic

Exchange; next active layer

Bitonic: pairs change between dependent layers. Each layer needs its measured signs before the next layer knows which original records occupy each position. The cycle repeats K times; source shuffle-only bypass steps do not add arithmetic noise.

1. Electronic

Gather all unordered pairs

2. Candidate photonic path

Encode / difference / read

3. Electronic

Sign and rank sums

4. Electronic

Validate; place original records

Rank: the unordered pairs can all be formed from the original input. Decisions increment electronic integer ranks. Records are placed only when the ranks form a permutation of $0..N-1$. A finite device may tile the pair phase; one wide logical stage is not a one-call latency guarantee.

The green arithmetic block in the diagrams is a proposed photonic operation. Every block is executed in software in this pilot. Encoding/readout and electronic control boundaries are explicit; no optical chaining is assumed.

8.1 Normalize keys without hiding their range

The user selected whole numbers including zero, with conventional ranges 0..15 and 0..255. A b -bit unsigned representation has 2^b levels and a maximum integer of $2^b - 1$. Invalid raw inputs are rejected before normalization.

```
M = 2^key_bits - 1; Lin = 2^input_bits.  
u(x) = x (1 - 1/Lin) / M.  
Qin(u) = rint(Lin*u) / Lin.
```

The maximum allowed key maps exactly to the largest unsigned input level, $1 - 1/Lin$. The scale is fixed by the declared domain, not the minimum and maximum in each test array. Every architecture receives the same scale.

Matched case	Normalization	Input step	Largest input level
4-bit key/input	$u = x/16$	$1/16$	$15/16$
8-bit key/input	$u = x/256$	$1/256$	$255/256$

For these matched cases, every integer key is represented exactly at the input: input quantization introduces no error. The experiment still records `input_bits` separately so future experiments can vary it independently. Input precision and output precision must not be conflated.

Why use this scale?

It uses the full representable input interval, preserves order because the scale is positive, and makes key spacing comparable across arrays in a case. It is a transparent experimental choice, not a universally optimal normalization proven by the supplied literature. Block floating point and data-dependent scaling are possible later alternatives, with additional scaling and execution costs.

```
validate_keys(raw_keys, key_bits)  
scale = (1 - 1 / input_levels) / maximum_key  
q = np.rint(raw_keys * scale * input_levels) / input_levels
```

8.2 Quantize the signed difference

A subtraction may be negative. The retained output model therefore uses a signed grid, whereas the input grid is unsigned. With L_{out} output levels, the interval is $[-1, 1-h]$ and h is one output quantization step.

```
Lout = 2^output_bits; h = 2/Lout.  
d = Qin(u(a)) - Qin(u(b)).  
epsilon ~ Normal(0, (eta*h)^2), eta = 0 or 0.25.  
d_measured = h * clip(rint((d+epsilon)/h), -Lout/2, Lout/2-1).
```

Operation	Reason / explicit convention
Divide by h	Express the noisy difference in output-code units.
rint	Choose the nearest integer code; exact half steps go to the even integer.
clip	Model output saturation if the rounded code is outside the signed converter range.
Multiply by h	Return the represented difference in normalized units.

Raw-key rejection and output saturation are different rules. A key 16 is rejected in the 4-bit domain. A valid measured subtraction can saturate at an output endpoint because of the signed grid or noise; this is counted as a numerical event, not silently treated as an input error.

The output grid, nearest-even rounding and exact digital operations are modeling conventions retained from the earlier pilot. They are not claimed to reproduce a Q.ANT ADC transfer function. Gaussian errors are independent across comparison events and trials; correlated, signal-dependent and stage-accumulating optical errors are outside this experiment.

```
codes = np.rint(noisy / output_step)  
result = np.clip(codes, -levels // 2, levels // 2 - 1)  
return result * output_step
```

8.3 Follow one comparison: keys 10 and 9

Use 8-bit keys, input quantization and output quantization. Both inputs are valid. The table first calculates quantization alone, then illustrates one possible noise sample. The sample is chosen to explain the arithmetic; it is not a fixed noise value used in the Monte Carlo run.

Step	No added noise	Example epsilon = +1/512
Raw keys	10, 9	10, 9
Normalize and quantize inputs	10/256, 9/256	10/256, 9/256
Difference d	1/256	1/256
Output step h	1/128	1/128
Add epsilon	1/256	3/512
Divide by h	0.5	0.75
Round to nearest even	0	1
Clip code to [-128,127]	0	1
Return code times h	0	1/128
Decision for [10,9]	Tie: earlier index first	10 is greater than 9
Two-record ascending output	[10,9], incorrect	[9,10], correct

Here $\sigma = 0.25h = 1/512$. Each simulated comparison draws its own sample, including negative samples. A sample $-1/512$ would leave a zero measured difference. The final value is a quantized comparison signal; original key values 10 and 9 remain unchanged.

The same issue at 4 bits

Inputs 10/16 and 9/16 differ by 1/16, but the output step is 1/8. The output code is again $\text{rint}(0.5)=0$ without noise. Matching input and output bit counts does not give matching input and signed-output resolution.

8.4 Bitonic: from source routing to code

Bitonic networks build short ascending and descending runs, then merge them through a fixed sequence of compare-exchanges. For $N=2^m$, the compact schedule has $K=m(m+1)/2$ active layers, each with $N/2$ pair comparisons. Beyette and Desmulliez implement a fixed perfect-shuffle schedule with $P=m^2-m+1$ processing steps, including bypasses. [5,6]

Source mechanism	Python representation	Reason
Optical perfect-shuffle wiring	Move values and original indexes to fixed destination indexes.	Presents the correct logical pairs to adjacent nodes.
Node difference / ordering decision	Difference(raw_a,raw_b); inspect sign and original indexes.	Proposed hybrid replacement for source comparison logic.
Ascending/descending node control	A stored schedule flag selects the exchange direction.	Creates and merges bitonic subsequences.
Electronic storage / recirculation	Update the current original records after each step.	Reuses the same logical node positions.
Bypass step	Permutation only; no Difference call.	Does not invent extra numerical comparisons.

For $i < N-1$: $\text{destination}(i) = 2i \bmod (N-1)$.

The last wire remains at $N-1$.

For $N=8$, $\text{destination}=[0,2,4,6,1,3,5,7]$. This is a wiring permutation, not a rank vector. The source example $[4,8,1,3,2,7,5,6]$ becomes $[4,8,3,1,2,7,6,5]$ after its first comparison step and $[1,2,3,4,5,6,7,8]$ after seven processing steps, six of them active.

Both named architectures run this schedule with identical numerical assumptions and matched error samples. Equal precision results are expected. Their physical node design, storage and historical timing remain distinct.

8.5 Bitonic: the exchange decision

```
d = difference(a, b) # RAW keys; model normalizes internally
greater = (d > 0) | ((d == 0) & (index_a > index_b))
exchange = np.where(ascending, greater, ~greater)
left = np.where(exchange, b, a)
right = np.where(exchange, a, b)
```

A positive measured difference places a after b in an ascending comparison. If the measured difference is zero, the original input index breaks the tie. For a descending node the order is reversed. Keys and their indexes are always exchanged together.

Compact reference layer	Example state for [7,2,5,1]
Input	[7,2,5,1]
1: adjacent pairs, opposite directions	[2,7,5,1]
2: distance-two pairs, ascending	[2,1,5,7]
3: adjacent pairs, ascending	[1,2,5,7]

This compact N=4 example explains the comparison layers; the evaluated fixed-shuffle implementation also carries out its explicit routing. Both representations agree ideally. Comparisons can be wrong while exchanges still preserve every record exactly once: the bitonic validity fraction is therefore always 100% in this model, even when the correct-sort fraction is low.

What is adapted

Historical optical interconnects and source node logic are not emulated at component level. The modern proposal retains a known sorting schedule but replaces its ordering information with a finite-precision pair-difference signal. This is why the experiment cannot be described as a hardware reproduction of either historical device.

8.6 Rank sorting: source matrix and modern adaptation

Louri broadcasts the input into a two-dimensional comparison structure. A difference matrix and a triangular bias resolve order and ties. Thresholded entries are summed by column to obtain one-based ranks, followed by output placement. The source includes smart-pixel electronic functions; it is not a purely linear optical MVM. [7]

For the reproduced source convention:

$\Delta_{ij} = x_j - x_i$.
 $\text{bias}_{ij} = 1$ if $i > j$, otherwise 0.
 $R_{ij} = 1$ if $\Delta_{ij} - \text{bias}_{ij} \geq 0$, otherwise 0.
 $\text{rank}_j = \sum_i R_{ij}$.

Source example	Result
Input	[7, 8a, 2, 8b, 5]
One-based ranks	[3,4,1,5,2]
Zero-based ranks	[2,3,0,4,1]
Stable output	[2,5,7,8a,8b]

The code validates this original matrix example separately. The numerical sweep then uses a deliberate adaptation: compare each unordered pair once, infer the complementary decision, sum integer ranks electronically and scatter original records electronically. There are $N(N-1)/2$ measured differences rather than N^2 independently evaluated matrix entries.

The source triangular unit bias is valid for its integer-key formulation. It is not blindly added as a unit offset to normalized analog values. The adaptation instead resolves a measured zero by original index. Ideal equivalence does not imply that its noisy physical behavior is identical to the paper.

8.7 Rank: count, validate, place

Unordered pair from [7,2,5,1]	Record receiving one rank increment
7 versus 2	7
7 versus 5	7
7 versus 1	7
2 versus 5	5
2 versus 1	2
5 versus 1	5

Final zero-based ranks: [3,1,2,0].

Place input record i at output position rank_i .

Output: [1,2,5,7].

```
left, right = np.triu_indices(n, 1)
d = difference(keys[:, left], keys[:, right])
left_is_larger = d > 0
# Every pair increments the rank of exactly one record.
# A measured tie favors the earlier original index.
valid = np.all(np.sort(ranks, axis=1) == np.arange(n), axis=1)
```

Original indices satisfy $\text{left} < \text{right}$, so $d=0$ assigns the increment to the later record. A rank counts how many records are judged to come before that record. The electronic sums use exact integers.

Why a rank output can fail structurally

Inconsistent comparisons can report $a > b$, $b > c$ and $c > a$. Then all three ranks can be 1. Three records request the same position, so there is no valid output permutation. Such a trial fails and is not repaired by a digital sort.

Distinct ranks can still describe an incorrect ordering. Consequently the experiment reports validity, correct sorting and stable sorting separately. Correctness is always divided by all 1,000 trials, including invalid ones.

9. What was validated against the papers

Source / locator	Reproduced	Adapted or unverified
Beyette [5], Eq.1; section 3.C	$P=m^2-m+1$. $N=8$, $w=3$ gives $P=7$ and $P+w-1=9$ cycles.	Comparison arithmetic replaced by shared Difference model. Physical timing, losses and hardware operation not measured.
Desmulliez [6], Fig.3; Eq.8	Eight-key trace; seven steps/six active layers. 1024 keys, $w=8$, $F=100\text{MHz}$: 9.1 us; about 900.22 Mbit/s.	Projected complete-system design point, not a measured 100MHz sorter. Identical numerical schedule to Beyette here.
Louri [7], Eqs.3-7; timing discussion	Original matrix ranks [3,4,1,5,2]. Reported component arithmetic: 60.2 ns full cycle, 36.8 ns period.	Sweep uses unordered pairs and electronic sums/placement. Detector estimate discrepancy remains unresolved.

Louri lists a detector estimate of about 13.33 ns; using the stated 10 pJ and 1.5 mW gives E/P about 6.67 ns. This factor-of-two issue is retained explicitly. Reproducing the timing sum does not resolve it or validate a measured complete sorter.

The validation file records these source calculations separately from the new precision rows. No historical optical transit time is copied into a Q.ANT latency prediction. Source PDF hashes identify the supplied versions used in the research.

What counts as evidence

A published equation reproduced by arithmetic establishes transcription and interpretation of that equation. A worked example establishes the corresponding functional behavior. Ideal checks establish correctness on the tested inputs. None alone establishes the physical speed, energy or accuracy of a modern mapping.

9.1 Independent checks before and during the sweep

Check	Actual test	Outcome
Source examples	Beyette equation; Desmulliez trace/operating point; Louri full matrix.	Pass; unresolved Louri physical discrepancy preserved.
Small-network wiring	Every binary input at N=2,4,8, for all three cases.	Correct and stable.
Every requested length	Random duplicates, endpoints, ascending, descending and all-equal arrays.	Ideal correct and stable.
ReLU identity	All 256 ordered pairs from 0..15.	Exact min/max identity passes; not an optical test.
Quantizer boundary control	All ordered pairs from each integer domain.	Input values exact; adjacent differences round to zero without noise.
Invalid inputs	Negative, too large, fractional, nonfinite, boolean and text keys.	Rejected with position and permitted range.
Noise reproducibility	Same trial streams executed together and in separate batches.	Identical outputs.
Entire ideal sweep	72 configurations, 72,000 architecture runs.	100% correct and stable.
Paired bitonic comparison	Every output, original index and validity mask.	Beyette and Desmulliez identical by design.

The two bitonic cases do not provide two independent sets of statistical evidence. Their paired outputs are checked, but confidence intervals are computed for each 1,000-trial configuration without pooling them.

Checks are finite. The mathematical sorting-network construction supports general ideal behavior; the numerical run only supports the declared distributions, grid conventions and noise settings.

10. The agreed experimental protocol

Parameter	Exact setting
Key domains	4 bits: all integers 0..15. 8 bits: all integers 0..255.
Input/output bit settings	(4,4) or (8,8), stored as input_bits and output_bits separately.
N, 4-bit case	2, 4, 8, 16
N, 8-bit case	2, 4, 8, 16, 32, 64, 128, 256
Distinct family	Uniform sample without replacement, randomized order. At maximum N this is a permutation of the whole domain.
Duplicates-allowed family	Independent uniform integer draws with replacement. An individual array need not actually contain duplicates.
Trials and inputs	1,000 arrays per (key_bits,N,family); exact same arrays used by all modes and architectures.
Conditions	Ideal; quantization only; quantization plus independent Gaussian noise, sigma=0.25 output steps.
Master seed	20260930. Full seed contexts and input hashes saved.
Digital operations	Exact comparisons of measured outputs, original-record movement, rank sums and placement.
Rank failures	Count as failed sorts; no digital correction.

There are 12 size/bit cases, two input families, three conditions and three named architectures: $12 \times 2 \times 3 \times 3 = 216$ configurations. Each has 1,000 runs: 216,000 architecture executions on 24,000 generated input arrays.

The comparison is between matched-width operating scenarios. As b increases, the domain expands as well as converter resolution. It does not isolate the causal effect of converter precision at a fixed key range.

10.1 Metrics and statistical interpretation

Metric	Definition and denominator
Correct-sort fraction	Valid output equals the exact ascending sorted input; count / all 1,000 trials.
Stable-sort fraction	Valid output record indexes equal a stable digital argsort; count / all trials.
Validity fraction	Output positions form a permutation. Bitonic always preserves records; rank can have collisions.
False-tie count	Number of comparison events with unequal raw keys but measured difference zero.
Sign-reversal count	Measured difference has a strict sign opposite to the true nonzero difference.
Output saturation count	Rounded output code exceeds a converter endpoint and is clipped.
95% Wilson interval	Binomial sampling uncertainty for correct-sort fraction within this exact experiment.

A comparison-event count is not a count of failed arrays: an array may contain many such events. Saturation need not change a sign. A false tie can be resolved correctly or incorrectly depending on the original record indexes. These diagnostics explain mechanisms; the complete-sort metric answers the task requirement.

NumPy sort provides ground truth only; it is not used to repair outputs and is not timed as a competitive CPU/GPU baseline. Kendall tau is not reported in this run: invalid rank outputs require an explicit treatment before an ordering-distance metric would be comparable.

For 0 correct outputs in 1,000 trials, the Wilson upper bound is about 0.383%. For 1,000 correct outputs it gives a lower bound about 99.617%. Neither observation proves a population probability exactly zero or one.

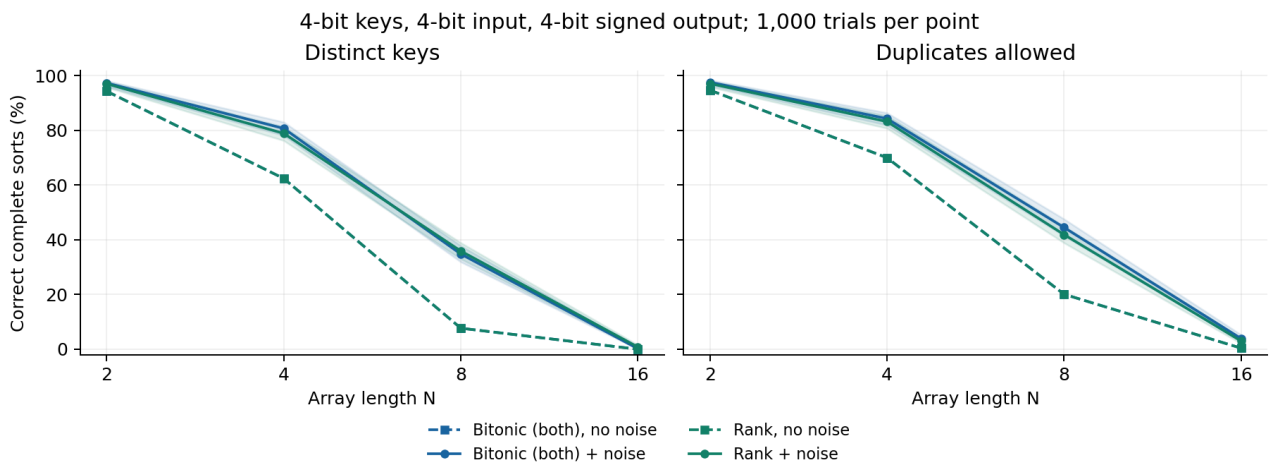
Confidence intervals do not capture unmodeled device effects or uncertainty in the numerical model. Small observed differences between algorithms do not establish superiority. A paired statistical comparison would use the saved per-trial outcomes and a declared hypothesis.

11. Results: what the new run establishes

Ideal correctness: every tested ideal case succeeds, including duplicates and the maximum lengths. The proposed functional mappings work when comparison information is exact.

Limited precision: adjacent keys can become false ties even without noise. Correct complete sorts become rare as N becomes large relative to the key domain. The raw integer range is now explicit and no accepted input is silently clipped.

Architecture distinction: the two bitonic cases give exactly the same results under the shared comparison model. Rank can additionally produce invalid output positions. The experiment identifies no universal winner.



Dashed: quantization only. Solid: quantization plus $\sigma=0.25$ output steps. Shading: 95% Wilson intervals for noisy points. Ideal cases are 100% throughout and omitted from the plot for readability. Both bitonic curves overlap by construction.

The exact tables on the next pages report correct runs out of 1,000. Complete three-architecture rows, intervals, stable counts and validity counts are available in the HTML inspector and `precision.csv`.

11.1 4-bit results: distinct keys

Raw keys 0..15; input_bits=4; output_bits=4. Every entry below is a count out of 1,000 trials. Bitonic denotes each of Beyette and Desmulliez; counts are not pooled.

N	Bitonic: no noise	Rank: no noise	Bitonic: noise	Rank: noise	Rank valid: noise
2	943	943	973	970	1000
4	624	624	807	789	997
8	77	77	348	359	941
16	0	0	2	8	544

Ideal arithmetic: 1,000/1,000 correct and stable for every N and every architecture in this table. Noise refers to independent Gaussian sigma=0.25 output steps.

Rank validity is shown separately from correctness. A valid permutation may still be incorrectly ordered. Invalid ranks remain in the denominator of the correct-sort fraction.

For duplicate keys, complete key order and stable record order are different requirements. The saved stable_sorts field tests the original record identities, not just the key values.

These data are sensitivity results for the declared numerical mapping. They are not measured Q.ANT sorting probabilities.

11.2 4-bit results: duplicates allowed

Raw keys 0..15; input_bits=4; output_bits=4. Every entry below is a count out of 1,000 trials. Bitonic denotes each of Beyette and Desmulliez; counts are not pooled.

N	Bitonic: no noise	Rank: no noise	Bitonic: noise	Rank: noise	Rank valid: noise
2	947	947	976	971	1000
4	700	700	843	832	990
8	201	201	445	419	845
16	4	4	39	30	338

Ideal arithmetic: 1,000/1,000 correct and stable for every N and every architecture in this table. Noise refers to independent Gaussian sigma=0.25 output steps.

Rank validity is shown separately from correctness. A valid permutation may still be incorrectly ordered. Invalid ranks remain in the denominator of the correct-sort fraction.

For duplicate keys, complete key order and stable record order are different requirements. The saved stable_sorts field tests the original record identities, not just the key values.

These data are sensitivity results for the declared numerical mapping. They are not measured Q.ANT sorting probabilities.

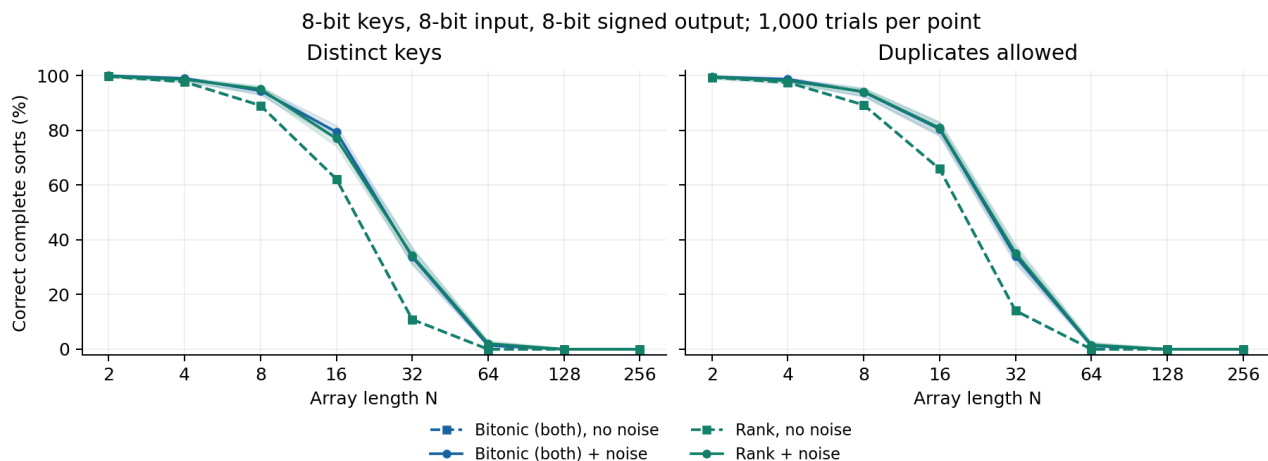
11.3 8-bit results: distinct keys

Raw keys 0..255; input_bits=8; output_bits=8. Every entry below is a count out of 1,000 trials. Bitonic denotes each of Beyette and Desmulliez; counts are not pooled.

N	Bitonic: no noise	Rank: no noise	Bitonic: noise	Rank: noise	Rank valid: noise
2	997	997	1000	998	1000
4	977	977	990	986	1000
8	891	891	945	951	1000
16	622	622	794	770	998
32	109	109	337	342	973
64	0	0	14	21	836
128	0	0	0	0	221
256	0	0	0	0	0

Ideal arithmetic: 1,000/1,000 correct and stable for every N and every architecture in this table. Noise refers to independent Gaussian $\sigma=0.25$ output steps.

Rank validity is shown separately from correctness. A valid permutation may still be incorrectly ordered. Invalid ranks remain in the denominator of the correct-sort fraction.



These data are sensitivity results for the declared numerical mapping. They are not measured Q.ANT sorting probabilities.

11.4 8-bit results: duplicates allowed

Raw keys 0..255; input_bits=8; output_bits=8. Every entry below is a count out of 1,000 trials. Bitonic denotes each of Beyette and Desmulliez; counts are not pooled.

N	Bitonic: no noise	Rank: no noise	Bitonic: noise	Rank: noise	Rank valid: noise
2	994	994	996	995	1000
4	975	975	987	981	1000
8	893	893	940	942	999
16	659	659	805	809	992
32	141	141	340	352	937
64	0	0	13	16	620
128	0	0	0	0	44
256	0	0	0	0	0

Ideal arithmetic: 1,000/1,000 correct and stable for every N and every architecture in this table. Noise refers to independent Gaussian $\sigma=0.25$ output steps.

Rank validity is shown separately from correctness. A valid permutation may still be incorrectly ordered. Invalid ranks remain in the denominator of the correct-sort fraction.

For duplicate keys, complete key order and stable record order are different requirements. The saved stable_sorts field tests the original record identities, not just the key values.

These data are sensitivity results for the declared numerical mapping. They are not measured Q.ANT sorting probabilities.

11.5 Read the results without overclaiming

Representative noisy case	Bitonic: correct / valid	Rank: correct / valid
4 bits, N=16, distinct	2/1000; 1000/1000	8/1000; 544/1000
8 bits, N=16, distinct	794/1000; 1000/1000	770/1000; 998/1000
8 bits, N=32, distinct	337/1000; 1000/1000	342/1000; 973/1000
8 bits, N=64, distinct	14/1000; 1000/1000	21/1000; 836/1000
8 bits, N=256, duplicates allowed	0/1000; 1000/1000	0/1000; 0/1000

Why noise sometimes raises the success fraction

An adjacent-key difference sits exactly at half an output step. With nearest-even rounding, it becomes zero deterministically without noise. A small positive or negative perturbation can move it into a nonzero code. This can resolve an otherwise misleading tie; the same noise can also create errors, including errors between truly equal keys.

This is a quantizer-boundary effect of the chosen model and input lattice. It does not support adding noise to a real sorter or treating Gaussian noise as physically beneficial in general.

An independent two-key control

Without noise, exhaustive enumeration of every ordered distinct pair gives a success probability of 93.75% for 4 bits and 99.609375% for 8 bits. The reason is simple: there are $L(L-1)$ ordered distinct pairs and $L-1$ descending adjacent pairs are misread as ties, giving failure probability $1/L$. This independently explains the small- N Monte Carlo behavior.

At maximum N , distinct arrays contain the entire domain, exposing every adjacent spacing. At a fixed small N , the 8-bit domain produces sparser random arrays than the 4-bit domain. Both density and numerical resolution must be considered when comparing their results.

In raw-key units: multiplying by the normalization denominator gives an output step of 2 integer-key units and noise $\sigma=0.5$ key units in both matched-width scenarios. Thus increasing both domain width and converter width does not improve adjacent-integer resolution in this model. A fixed-domain converter sweep is needed to isolate that effect.

11.6 Resource counts and timing boundaries

N	Active K	Shuffle P	Bitonic differences	Rank differences
2	1	1	1	1
4	3	3	6	6
8	6	7	24	28
16	10	13	80	120
32	15	21	240	496
64	21	31	672	2016
128	28	43	1792	8128
256	36	57	4608	32640

The counts are per input array. Bitonic uses $N/2$ differences in each of K dependent active layers. Rank uses $N(N-1)/2$ differences followed by electronic accumulation and placement. P counts the researched shuffle processing steps, including bypasses; it is not a count of noisy subtraction rounds.

For $N=256$, the bitonic mapping evaluates 4,608 differences in 36 active layers and the rank mapping evaluates 32,640 pair differences. Rank exposes more independent work; whether that offsets its larger work and data movement depends on the device.

What is not a measured timing result

No Q.ANT latency or throughput is inferred from NumPy execution time. Source clocks, vendor GOPS and optical transit times have different boundaries. A modern timing model must establish batching, supported dimensions, dispatch, host/device transfers, output conversion, electronic decisions and memory costs.

The simulator stores all pairs in a batch for convenience. Its Python memory use and vectorization are not measurements of required physical photonic area or an optimized hardware implementation. The earlier C table is structural accounting under its stated conventions.

12. Next experiments and hardware questions

Priority	Question	Evidence needed
1. Numerical control	How much failure comes from signed output resolution?	Keep the key range and exact input arrays fixed; vary input_bits and output_bits independently, then compare the same metrics.
2. Candidate kernel	Can the [1,-1] pair kernel run efficiently?	Accepted dimensions, ranges, batching/tiling, weight loading and measured sign-error behavior.
3. Timing boundary	What dominates a complete sort?	Dispatch, transfer, encode/readout, device arithmetic and electronic control timings under one stated boundary.
4. Optical nonlinearity	Can a useful comparator/min-max be implemented?	Documented transfer function, operating interval, precision and gain; do not substitute a function name for device evidence.
5. Regeneration interval	How many nonlinear stages can be cascaded?	Calibrated noise/loss propagation and actual optical chaining support; otherwise explicitly hypothetical parameters.
6. Energy	Does either mapping have a system advantage?	Measured or justified power/event energies with the same correctness and system boundaries as the digital baseline.

The first follow-up should separate key width from signed output resolution. Merely repeating more Monte Carlo trials cannot resolve a systematic quantizer limitation. The current run is useful evidence for choosing that experiment.

A future bitonic cascade may use $r=\max(a-b,0)$, $\min=a-r$ and $\max=b+r$. This is an exact mathematical identity. It requires a defensible nonlinear physical operation and signal path before a regeneration-interval model can claim hardware relevance.

PRISM remains related work on photonic similarity scoring followed by electronic selection. Full sorting, stable sorting and top-k retrieval must have separate task definitions and baselines. [12]

13. Thesis registration: a concrete research goal

Proposed title

Hardware-Aware Evaluation of Bitonic and Rank Sorting on Hybrid Photonic Accelerators

Goal

Establish under which documented or explicitly parameterized precision, resource and execution conditions a bitonic or rank-based mapping can perform correct full sorting on a hybrid photonic platform. Use Q.ANT as a reference where its public evidence supports the mapping. Do not assume an efficiency advantage.

Research question	Current evidence / next contribution
How does the proposed C model relate to earlier models?	Preserved historical comparison and five-architecture derivations.
Do the proposed mappings sort ideally?	Selected paper examples, independent checks and every ideal sweep case pass.
Where do they fail numerically?	False ties from output resolution, noise-related comparison errors, and inconsistent ranks.
Which algorithm is preferable under specified conditions?	Compare at the same correctness requirement and documented resource/timing boundary. No universal winner established.
How do nonlinearity and regeneration change the result?	Central next experiment, contingent on defensible optical functions and cascade assumptions.

The registration milestone is a reproducible functional comparison and a precise hardware evidence gap. An energy-efficiency verdict follows only after mapping, timing and power evidence are sufficient. A negative result can still identify useful limits and conditions for photonic sorting.

Deliverables now available: documented code, complete configuration, saved inputs and outcomes, all result rows, source-validation records, operation counts, this updated PDF and an offline interactive HTML report.

14. Reproduce and inspect the evaluation

Extract `qant_sorting_pilot.zip` and open its `qant_sorting_pilot` folder in VS Code. Python 3.12 and the pinned dependencies reproduce the tested software environment. The included results can be read without rerunning.

Windows PowerShell

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe validate.py
.\.venv\Scripts\python.exe evaluate.py
.\.venv\Scripts\python.exe build_reports.py
```

Linux / macOS

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python validate.py
.venv/bin/python evaluate.py
.venv/bin/python build_reports.py
```

Choose the project `.venv` with VS Code: Python: Select Interpreter. The commands above use that environment explicitly, so shell activation is optional. Read `README.md` for the entry points and example input.

`evaluate.py` overwrites its output folder. To retain the supplied results while experimenting, pass `--output results_trial` and a separate `--config` path. `build_reports.py` reads the canonical results folder. Only the accepted (4,4) and (8,8) settings are reported in this revision.

The HTML report works offline and contains all 216 rows. Its selectors change the view only; they do not start simulations or fabricate predictions. The worked quantizer example is explicitly illustrative.

14.1 Configuration, seeds and the evidence files

File	Meaning
config.json / results/config.json	Requested sweep and the saved run configuration.
results/precision.csv	216 rows: exact counts, fractions, Wilson intervals, precision settings and seed contexts.
results/inputs.npz	24 datasets of 1,000 arrays; identified by bit width, N and family.
results/trial_outcomes.npz	One matrix per case: columns correct, stable, valid. Rows are trial indexes.
results/failures.json	First incorrect trial per failing case, with input, expected output, actual output and record indexes.
results/resources.csv	Per-array pair counts, active rounds and source shuffle steps.
results/validation.json	Source examples, mathematical controls, input errors and full-sweep checks.
results/run_metadata.json	Software versions, code hashes and runner wall time, explicitly not hardware latency.
results/source_hashes.json	SHA-256 fingerprints of the supplied literature PDFs.

Input seeds use [master_seed,key_bits,N,family_id,0]. Noise seeds use [master_seed,key_bits,input_bits,output_bits,N,family_id,mode_id,stream_id,trial_index]. Families and modes follow config.json order. stream_id=1 for both bitonic cases and 2 for rank.

Each trial has an independent NumPy generator. Consequently changing the memory batch size does not change the same trial sequence. The different algorithm schedules do not use identical per-pair noise samples: they use the same noise law, independent between rank and bitonic. The two bitonic cases intentionally share all samples.

This run supersedes the previous N=8..64, spacing=1/4, 6..9-bit experiment as the current precision evidence. Old source examples and theory remain relevant; old result counts and input assumptions must not be combined with the new rows.

15. Glossary and evidence boundary

Term	Meaning
Key / record	A sortable value / the value together with its original identity or payload.
Stable sort	Equal true keys preserve their original order.
False tie	Different raw keys produce a measured zero difference.
Quantization step / LSB	Spacing between adjacent represented levels of the specified converter.
Signed output	A difference representation admitting negative, zero and positive values.
Normalization	Positive fixed scaling from the declared integer domain to an input signal range.
Saturation	A rounded converter code outside the range is limited to an endpoint.
Noise sigma	Standard deviation of the declared random error, here in output-step units.
Rank collision	Multiple records request the same output position.
Monte Carlo / seed	Repeated random trials / recorded initialization for reproducing them.
Wilson interval	Sampling uncertainty for an observed success fraction; not device uncertainty.
ENOB / BF16	Measured effective precision / a software floating-point storage format.
Regeneration interval	Number of stages between signal restoration operations; not varied in this run.
Optical depth D	Maximum unregenerated optical stages under the framework definition.
Sensitivity / calibration	Vary declared assumptions / fit or verify against hardware measurements.

This pilot is a sensitivity study. Its assumptions and limitations are part of its result: it establishes a working functional baseline and exposes the precision questions that hardware evidence must address.

15.1 References

- [1] Thompson (1979). Area-Time Complexity for VLSI.
<https://doi.org/10.1145/800135.804401>
- [2] Caulfield (1991). Space-time complexity in optical computing.
<https://doi.org/10.1007/BF01937172>
- [3] Li and Monticone (2025). The spatial complexity of optical computing.
<https://doi.org/10.1038/s41467-025-63453-8>
- [4] Stirk and Athale (1988). Sorting with optical compare-and-exchange modules. Passive/restoring variants: p.1725.
<https://doi.org/10.1364/AO.27.001721>
- [5] Beyette et al. (1994). Bitonic sorting using an optoelectronic recirculating architecture. Eq.1, p.8165; section 3.C, p.8171.
<https://doi.org/10.1364/AO.33.008164>
- [6] Desmulliez et al. (1995). Perfect-shuffle interconnected bitonic sorter: optoelectronic design. Fig.3, p.5080; Eq.8 and projected design point, pp.5085-5088.
<https://doi.org/10.1364/AO.34.005077>
- [7] Louri et al. (1995). Constant-time parallel sorting algorithm and its optical implementation using smart pixels. Eqs.3-7; timing pp.3093-3094.
<https://doi.org/10.1364/AO.34.003087>
- [8] Mizutani et al. (2025; online 2024). In-network stable radix sorter using many FPGAs with high-bandwidth photonics.
<https://doi.org/10.1364/JOCN.530695>

Primary supplied literature and the preceding inspected hardware snapshot support the architectural account. Experimental settings and new results are explicitly identified as such. Source PDFs are not redistributed in the project ZIP or repository.

15.2 Hardware and numerical background

[9] Q.ANT photonic-computing product page. Vendor snapshot checked 1 October 2026; no new hardware measurement.

<https://qant.com/photonic-computing/>

[10] Q.ANT Native Computing Toolkit. Inspected commit 72a2d99f10240b6df3c6d0f636dfa0e2b5d38902.

https://github.com/Q-ANT-GmbH/qant_native_computing_toolkit

[11] LRZ. Photonic computers in real-world test. Separate operating context; conversion bits are not ENOB.

<https://www.lrz.de/en/news/detail/photonic-computers-in-real-world-test>

[12] PRISM (2026). Photonic Similarity Engine for KV Cache Block Selection in Long-Context LLM Inference.

<https://doi.org/10.48550/arXiv.2603.21576>

[13] The Physics of Optical Computing (2023).

<https://doi.org/10.1038/s42254-023-00645-5>

[14] Noise in analog programmable-photonic computation (2026). Background for later physical noise modeling, not the source of $\sigma=0.25$.

<https://doi.org/10.48550/arXiv.2604.24541>

[15] Energy Efficiency in Analog Photonic Processors: Conversions and Losses at Scale (2026). System-boundary motivation.

<https://doi.org/10.23919/ISC.2026.11520497>

Primary supplied literature and the preceding inspected hardware snapshot support the architectural account. Experimental settings and new results are explicitly identified as such. Source PDFs are not redistributed in the project ZIP or repository.